

CIT4114 ASSIGNMENT: WRITTEN REPORT

Program Design

TC2L GROUP NO: 17

NO	STUDENT ID	STUDENT NAME	HANDPHONE
1	243DC2455E	PRAVIN A/L KANNATHASAN	016-950 8191
2	243DC2460W	KISHVEER RAJ A/L ETHURAJOO	011-3769 1269

TO

LECTURER NAME	DR. AZIAH BINTI ALI
---------------	---------------------

MULTIMEDIA UNIVERSITY

TRIMESTER (2510) **CCP4114-PROGRAM DESIGN**

TABLE OF CONTENTS

NO.	CONTENTS	PAGE
1	Introduction	3
2	Program Description	4
3	Codes Explanation	5
4	Source Code	6-11
5	Sample Output	12-14
6	Task division and contributions	15
7	Evaluation	16-18

INTRODUCTION

A command line application called the Student Academic Planner was created to assist students in managing their academic obligations. Important information like the student ID, name, course name, assignment title, due date, and completion status can be stored and managed by users. Students can add new tasks, view all entries, search for particular records, edit information, and remove completed or out-of-date tasks using an easy-to-use menu system. The planner is a useful and dependable tool for academic organization since it guarantees that all data is saved to a file and is accessible even after the program has ended.

PROGRAM DESCRIPTION

Users can add new planner entries, view all existing records, search for specific student entries, edit previously saved data, and delete records when they are no longer needed using the application's straightforward, menu-driven interface. Every time the program is launched, all data is automatically loaded from a CSV file to guarantee data persistence.

The Student Academic Planner offers a useful way to manage academic obligations by managing input validation, structured data management, and real-time modification saving. Students who prefer a lightweight, terminal-based system to keep track of their academic deadlines and assignments will find it especially helpful.

CODE EXPLANATION

This section explains the main components and logic behind the *Student Academic Planner C* program.

1. Data Structure (PlannerEntry)

The program defines a structure named PlannerEntry to store each academic task:

```
c
CopyEdit
typedef struct {
    char studentID[20];
    char studentName[50];
    char courseName[50];
    char assignmentTitle[50];
    char dueDate[20];
    char status[20];
} PlannerEntry;
```

Each record includes student details, course info, assignment title, due date, and completion status.

2. Login System

```
c
CopyEdit
int login()
```

- The program starts with a login screen.
 - Accepts hardcoded credentials (student / planner).
 - Allows up to **3 attempts** before exiting.
-

3. Menu Navigation

c

CopyEdit

void menu(PlannerEntry entries[], int *count)

- Displays a **menu with 6 options**:
 1. Add New Entry
 2. View All Entries
 3. Delete an Entry
 4. Search an Entry
 5. Edit an Entry
 6. Exit
 - Each menu choice calls a specific function.
-

4. File Handling

- **planner.csv** stores all data for persistence.
 - On startup, loadData() reads the file.
 - After changes (add, edit, delete), saveData() updates the file.
-

5. Core Functions

addEntry()

- Prompts the user to enter data for all fields.
- Validates non-empty input.
- Saves the new entry and updates the file.

viewEntries()

- Displays all planner records in a formatted table.
- Handles cases where no records exist.

deleteEntry()

- Deletes a record based on **Student ID**.
- Uses a simple array shift to remove the entry.

searchEntry()

- Searches for a record by **Student ID**.
- Displays the full details if found.

edit Entry()

- Searches for a record and allows the user to update all fields.
 - Overwrites the old data and saves changes.
-

6. Input Validation

- `getValidatedInt()` ensures valid menu input (1–6).
 - `getValidatedString()` ensures no empty inputs for user-entered strings.
 - `trimNewline()` removes newline characters from `fgets()` input.
-

7. Supporting Functions

- `clearInputBuffer()` avoids issues from leftover input.
 - `findIndexByID()` is used in search, delete, and edit operations to locate records quickly.
-

8. Modularity and Structure

- The program uses modular design with clear function separation.
- This improves **readability**, **reusability**, and **debugging**.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_ENTRIES 100
#define FILENAME "planner.csv"
#define MAX_ATTEMPTS 3

// Define the structure for an academic planner entry
typedef struct {
    char studentID[20];
    char studentName[50];
    char courseName[50];
    char assignmentTitle[50];
    char dueDate[20];
    char status[20];
} PlannerEntry;

// Function Prototypes
int login();
void menu(PlannerEntry entries[], int *count);
void loadData(PlannerEntry entries[], int *count);
void saveData(PlannerEntry entries[], int count);
void addEntry(PlannerEntry entries[], int *count);
void viewEntries(const PlannerEntry entries[], int count);
void deleteEntry(PlannerEntry entries[], int *count);
void searchEntry(const PlannerEntry entries[], int count);
void editEntry(PlannerEntry entries[], int count);
void clearInputBuffer();
void trimNewline(char *str);
int findIndexByID(const PlannerEntry entries[], int count, const char *id);
int getValidatedInt(int min, int max);
void getValidatedString(char *prompt, char *input, int size);

// Entry point
int main() {
    PlannerEntry entries[MAX_ENTRIES];
    int count = 0;

    loadData(entries, &count);

    printf("-----\n");
```

```

printf("|                STUDENT ACADEMIC PLANNER                |\n");
printf("-----\n");

if (login()) {
    menu(entries, &count);
}

return 0;
}

// Login Function
int login() {
    char username[20], password[20];
    const char *correctUsername = "student";
    const char *correctPassword = "planner";
    int attempt = 0;

    while (attempt < MAX_ATTEMPTS) {
        printf("\nUsername: ");
        scanf("%19s", username);
        clearInputBuffer();

        printf("Password: ");
        scanf("%19s", password);
        clearInputBuffer();

        if (strcmp(username, correctUsername) == 0 && strcmp(password,
correctPassword) == 0) {
            printf("Login successful. Welcome!\n");
            return 1;
        } else {
            printf("Invalid credentials. Attempts left: %d\n",
MAX_ATTEMPTS - attempt - 1);
        }
        attempt++;
    }

    printf("Too many failed attempts. Exiting program.\n");
    return 0;
}

// Load data from CSV file
void loadData(PlannerEntry entries[], int *count) {
    FILE *file = fopen(FILENAME, "r");
    if (!file) return;

    while (fscanf(file,
"%19[^,],%49[^,],%49[^,],%49[^,],%19[^,],%19[^\n]\n",
entries[*count].studentID,
entries[*count].studentName,
entries[*count].courseName,
entries[*count].assignmentTitle,
entries[*count].dueDate,
entries[*count].status) == 6) {
        (*count)++;
    }

    fclose(file);
}

```

```

}

// Save data to CSV file
void saveData(PlannerEntry entries[], int count) {
    FILE *file = fopen(FILENAME, "w");
    if (!file) {
        perror("File write error");
        return;
    }

    for (int i = 0; i < count; i++) {
        fprintf(file, "%s,%s,%s,%s,%s,%s\n",
            entries[i].studentID, entries[i].studentName,
            entries[i].courseName, entries[i].assignmentTitle,
            entries[i].dueDate, entries[i].status);
    }

    fclose(file);
}

// Main menu function
void menu(PlannerEntry entries[], int *count) {
    int choice;

    do {
        printf("\n----- Main Menu ----- \n");
        printf("1. Add New Entry\n");
        printf("2. View All Entries\n");
        printf("3. Delete an Entry\n");
        printf("4. Search an Entry\n");
        printf("5. Edit an Entry\n");
        printf("6. Exit\n");
        printf("----- \n");
        printf("Enter your choice (1-6): ");

        choice = getValidatedInt(1, 6);

        switch (choice) {
            case 1: addEntry(entries, count); break;
            case 2: viewEntries(entries, *count); break;
            case 3: deleteEntry(entries, count); break;
            case 4: searchEntry(entries, *count); break;
            case 5: editEntry(entries, *count); break;
            case 6: printf("Exiting program. Goodbye!\n"); break;
            default: printf("Invalid choice. Try again.\n");
        }

    } while (choice != 6);
}

// Add a new planner entry
void addEntry(PlannerEntry entries[], int *count) {
    if (*count >= MAX_ENTRIES) {
        printf("Planner is full. Cannot add more entries.\n");
        return;
    }

    PlannerEntry *e = &entries[*count];

```

```

        getValidatedString("Enter Student ID: ", e->studentID, sizeof(e-
>studentID));
        getValidatedString("Enter Student Name: ", e->studentName,
sizeof(e->studentName));
        getValidatedString("Enter Course Name: ", e->courseName, sizeof(e-
>courseName));
        getValidatedString("Enter Assignment Title: ", e->assignmentTitle,
sizeof(e->assignmentTitle));
        getValidatedString("Enter Due Date (DD/MM/YYYY): ", e->dueDate,
sizeof(e->dueDate));
        getValidatedString("Enter Status (Pending/Done): ", e->status,
sizeof(e->status));

        (*count)++;
        saveData(entries, *count);
        printf("Entry added successfully.\n");
    }

// Display all entries
void viewEntries(const PlannerEntry entries[], int count) {
    if (count == 0) {
        printf("No records to display.\n");
        return;
    }

    printf("\n%-10s | %-15s | %-10s | %-15s | %-12s | %-10s\n",
        "ID", "Name", "Course", "Assignment", "Due Date", "Status");
    printf("-----\n");

    for (int i = 0; i < count; i++) {
        printf("%-10s | %-15s | %-10s | %-15s | %-12s | %-10s\n",
            entries[i].studentID, entries[i].studentName,
            entries[i].courseName, entries[i].assignmentTitle,
            entries[i].dueDate, entries[i].status);
    }
}

// Delete entry by student ID
void deleteEntry(PlannerEntry entries[], int *count) {
    char id[20];
    getValidatedString("Enter Student ID to delete: ", id, sizeof(id));

    int index = findIndexByID(entries, *count, id);
    if (index == -1) {
        printf("Record not found.\n");
        return;
    }

    for (int i = index; i < *count - 1; i++) {
        entries[i] = entries[i + 1];
    }

    (*count)--;
    saveData(entries, *count);
    printf("Record deleted successfully.\n");
}

```

```

// Search entry by student ID
void searchEntry(const PlannerEntry entries[], int count) {
    char id[20];
    getValidatedString("Enter Student ID to search: ", id, sizeof(id));

    int index = findIndexByID(entries, count, id);
    if (index == -1) {
        printf("Record not found.\n");
        return;
    }

    PlannerEntry e = entries[index];
    printf("\nStudent ID: %s\nName: %s\nCourse: %s\nAssignment: %s\nDue
Date: %s\nStatus: %s\n",
        e.studentID, e.studentName, e.courseName, e.assignmentTitle,
e.dueDate, e.status);
}

// Edit entry by student ID
void editEntry(PlannerEntry entries[], int count) {
    char id[20];
    getValidatedString("Enter Student ID to edit: ", id, sizeof(id));

    int index = findIndexByID(entries, count, id);
    if (index == -1) {
        printf("Record not found.\n");
        return;
    }

    PlannerEntry *e = &entries[index];

    printf("\nEditing details for %s:\n", e->studentID);
    getValidatedString("New Student Name: ", e->studentName, sizeof(e-
>studentName));
    getValidatedString("New Course Name: ", e->courseName, sizeof(e-
>courseName));
    getValidatedString("New Assignment Title: ", e->assignmentTitle,
sizeof(e->assignmentTitle));
    getValidatedString("New Due Date: ", e->dueDate, sizeof(e-
>dueDate));
    getValidatedString("New Status: ", e->status, sizeof(e->status));

    saveData(entries, count);
    printf("Record updated successfully.\n");
}

// Utility Functions

// Get integer input with validation
int getValidatedInt(int min, int max) {
    int value;
    while (1) {
        if (scanf("%d", &value) != 1 || value < min || value > max) {
            clearInputBuffer();
            printf("Invalid input. Enter number between %d and %d: ",
min, max);
        } else {

```

```

        clearInputBuffer();
        return value;
    }
}

// Get string input and trim newline
void getValidatedString(char *prompt, char *input, int size) {
    printf("%s", prompt);
    fgets(input, size, stdin);
    trimNewline(input);

    while (strlen(input) == 0) {
        printf("Input cannot be empty. %s", prompt);
        fgets(input, size, stdin);
        trimNewline(input);
    }
}

// Remove newline from string
void trimNewline(char *str) {
    str[strcspn(str, "\n")] = '\0';
}

// Find entry index by student ID
int findIndexByID(const PlannerEntry entries[], int count, const char
*id) {
    for (int i = 0; i < count; i++) {
        if (strcmp(entries[i].studentID, id) == 0)
            return i;
    }
    return -1;
}

// Clear leftover input
void clearInputBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

```

SAMPLE OUTPUT

Welcome page & Login form

```
|          STUDENT ACADEMIC PLANNER SYSTEM          |
```

```
Username: student  
Password: planner  
Login successful. Welcome!
```

Main menu

```
----- Main Menu -----  
1. Add Entry  
2. View All Entries  
3. Delete Entry  
4. Search Entry  
5. Edit Entry  
6. Exit
```

Add Entry

```
Student ID: 243DC2455E
Student Name: Pravin Kannathasan
Course Name: Program Design
Assignment Title: Program Design Assignment
Due Date (DD/MM/YYYY): 29/6/2025
Status (Pending/Done): Done
Entry added successfully.
```

View All Entry

ID	Name	Course	Assignment	Due Date	Status
1234	1234	1234	1234	0	
243DC2455E	Pravin Kannathasan	Program Design	Program Design Assignment	29/6/2025	Done

Delete Entry

```
Enter Student ID to delete: 1234
Entry deleted.
```

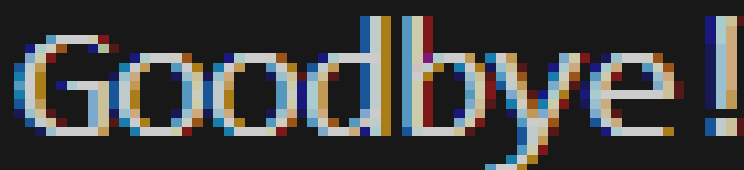
Search Entry

```
Enter Student ID to search: 243DC2455E
Student ID: 243DC2455E
Name: Pravin Kannathasan
Course: Program Design
Assignment: Program Design Assignment
Due Date: 29/6/2025
Status: Done
```

Edit Entry

```
Enter Student ID to edit: 243DC2455E
New Student Name: Pravin A/L Kannathasan
New Course Name: Program Design
New Assignment Title: Program Design Assignment
New Due Date: 29/6/2025
New Status: Pending
Entry updated.
```

Exit



Goodbye!

Task division and Contributions

Student ID	Student Name	Task Contribution
243DC2455E	PRAVIN A/L KANNATHASAN	Login system, Add Entry, Save Data, menu, View Entries, Delete Entry
243DC2460W	KISHVEER RAJ A/L ETHURAJOO	Search Entry, Edit Entry, Input validation, File Handling, Testing, Written Report

Evaluation Form

MARKING SCHEME

STUDENT ID: 243DC2455E, 243DC2460W

STUDENT NAME: PRAVIN A/L KANNATHASAN, KISHVEER RAJ A/L ETHURAJOO

LECTURE SESSION : TT2L

GROUP NUMBER: G17

CORRECTNESS OF THE PROGRAM AND REQUIREMENTS IMPLEMENTATION (40%)

Criteria	Exceeds Requirement s (4)	Meets Requirement s (3)	Approaching Requirement s (2)	Below Requirement s (1)
Core Features Implementatio n				
Data Types, Variables & Expressions				
Program flow				
Program execution				
I/O formatting – displays, prompts, input taking				
Total				

USE OF APPROPRIATE PROGRAMMING CONCEPTS (30%)

Criteria	Exceeds Requirements (4)	Meets Requirements (3)	Approaching Requirements (2)	Below Requirements (1)
Control Structures & Functions				
Arrays, Pointers & Structures				
File Handling				
Total				

CODE STRUCTURE AND READABILITY (10%)

Criteria	Exceeds Requirements (4)	Meets Requirements (3)	Approaching Requirements (2)	Below Requirements (1)
Code Structure and Readability Comments usage				
Total				

REPORT AND PRESENTATION (20%)

Criteria	Exceeds Requirements (4)	Meets Requirements (3)	Approaching Requirements (2)	Below Requirements (1)
Report				
Presentation (INDIVIDUAL MARK)				
Total				